FitBuddy-AI Fitness Plan Generator using Gemini Models

Phase 2 – Requirement Analysis

1. Introduction

Requirement analysis converts the project idea into clearly defined software requirements. For FitBuddy, the analysis identifies what information the system must collect, what processing it must perform, what outputs it must produce, and what technical environment is required. This phase also separates functional requirements from non-functional requirements so that development and testing can be organized properly.

2. Functional Requirements

The application must provide a home page through which users can enter their details. The required information includes username, user ID, age, weight, goal, and intensity. The backend must accept this information using form data and validate the values through the application schema.

After valid submission, the system must generate a workout plan using the Gemini AI module. It must also generate a nutrition tip through the nutrition-generation module. The generated data must be stored in the database. If the user ID already exists, the application must update the existing user's information and replace the original plan with the newly generated plan.

The application must accept feedback for a stored user. It must verify that the user exists, pass the original plan and feedback to the update module, save the revised plan, and display the updated result. The system must also provide administrator operations for viewing all users and deleting a selected user after checking the administrator key.

Finally, a health endpoint must be available to confirm that the backend service is running.

3. Non-Functional Requirements

The user interface should be simple and understandable. The application should respond reliably to valid requests and should provide useful error messages when an operation fails. The code should be separated into logical modules so that changes to database operations, routes, AI generation, and configuration can be made independently.

Administrative functions should not be openly accessible without the required key. The application should also maintain data consistently when an existing user is updated. The project should be easy to run in a Python virtual environment and should provide API documentation through FastAPI's documentation interface.

4. Input and Output Requirements

The main input fields are username, user ID, age, weight, goal, and intensity. Feedback is a separate input for the plan revision process. The administrator operations require an administrator key.

The main output is an AI-generated workout plan. A nutrition tip is also produced. The result page should display the stored user information and generated content. After feedback, the revised plan should be available. Administrative requests return user information or perform deletion. The health endpoint returns a simple status response indicating that the service is running.

5. Software and Technology Requirements

The project uses Python as the primary programming language. FastAPI provides the web API and routing layer, while Uvicorn runs the application server. SQLAlchemy provides database interaction and SQLite is used for local data storage. Jinja2 templates provide HTML rendering. The python-multipart package is required because the application accepts HTML form data. The Google GenAI/Gemini SDK is used for AI generation, and environment configuration is handled through the project configuration module and dotenv support.

6. Hardware and Environment Requirements

The current prototype can be developed and demonstrated on a normal computer capable of running Python. Internet connectivity is required when the application communicates with the Gemini service. A modern web browser is required to access the application and Swagger documentation. A Python virtual environment is recommended so that project dependencies remain isolated from the system Python installation.

7. Conclusion

The requirement analysis phase defines the expected behavior and technical environment of FitBuddy. These requirements provide the reference for system design, development, testing, and final demonstration. A clear requirement list also makes it easier to identify whether each feature has been implemented successfully.

Phase Deliverables / Evidence

Item	Purpose	Status / Evidence
Functional requirements	Defines system behavior	Documented
Non-functional requirements	Defines quality expectations	Documented
Technology requirements	Defines implementation stack	Documented